

CESE5040

High-Performance and AI Architectures

2026 Q4

Student Information

Name: Daniel Tyukov

Student Number: 5714699

Lab: Lab Assignment 2

Notes

- All timings are from the same AWS course VM: Intel Xeon Platinum 8375C at 2.90 GHz, 4 physical cores (8 vCPUs with SMT), 15 GiB RAM, Ubuntu 24.04, Python 3.12.3, NumPy 2.4.4, SciPy 1.17.1.
- Each number is the minimum of 3 runs.
- Code submitted: `L2_Q6.py`, `L2_Q7.py`, `L2_Q8.py`. The assignment lists 2.6, 2.7, 2.9 but 2.9 is text only, so I'm submitting 2.8 instead since that is where the multithreading code actually lives.

Lab Exercise 2

Question 2.1 — Multiprocessing challenges

There are two dependencies in TVB that fight each other when you try to parallelise it:

- Across timesteps, $X_j(t - d_{ij})$ has to be readable for every j before step $t + 1$ can run, so the next timestep can't start until the current one has finished writing its slice of X s. There has to be a barrier per timestep. With $T = \text{tf}/\text{dt}$ timesteps that's a lot of barriers.
- Within one timestep the N centres only read earlier slices of X s and write disjoint rows, so they are independent and can run in parallel. That's the parallelism we can actually use.

Beyond that the practical issues are:

- Communication: Python multiprocessing serialises every argument and every return value. X s has shape $N \cdot M \cdot T$ doubles (≈ 5.5 MB for TVB192 at 60 ms, ≈ 24 MB for TVB998 at 15 ms) and every task needs it. Copying it every timestep is what kills the naive version (see Q2.2.5).
- Task size: one centre at one timestep is one MLP call plus one N -element sum. That's tiny compared to the cost of submitting a task, so you have to batch with a non-trivial chunksize (Q2.2.3).
- Pure-Python state: `tvb_par.py` stores X s as a triply-nested list. The serialiser walks every scalar individually instead of copying a flat block. Q2.3 fixes that with NumPy in shared memory.

Question 2.2 — tvb_par.py analysis

1. What if the two lines were swapped (loop outside the *with*, executor inside)? A fresh `ProcessPoolExecutor` would be created and shut down on every timestep. Spinning up the pool forks all the workers, has them re-import the module, and then tears them all down again. That's exactly the cost the pool was supposed to avoid by living through the whole simulation. For $T = 300$ timesteps at TVB76 the pool is built once today; with the swap it would be built 300 times. Expected result: a slowdown of one to two orders of magnitude. Numerical output stays the same.

2. Compare to sequential MLP at 15 ms.

Dataset	Sequential MLP (s)	Multiprocess, default chunk=40 (s)	Speed-up
TVB76	0.77	1.48	0.52×
TVB192	2.56	6.16	0.42×

Table 1: Sequential vs naïve multiprocess TVB-MLP at $t_f=15$ ms. Minimum of 3 runs on the AWS VM (8 vCPUs).

Multiprocessing is *slower* than sequential at this scale. The reason is the IPC cost. Every `executor.map` call serialises the full nested-list X s (once per task), one row of W , one row of $D_timestep$, plus a couple of scalars. The worker decodes all of that, does $\mathcal{O}(N)$ of pure-Python scalar work (which was already cheap), and sends the result back. With 4 physical cores the best-case speedup is $4\times$, but the per-message cost of moving kilobytes around dominates the microseconds of real work, so the parallel version loses.

3. Chunk-size sweep.

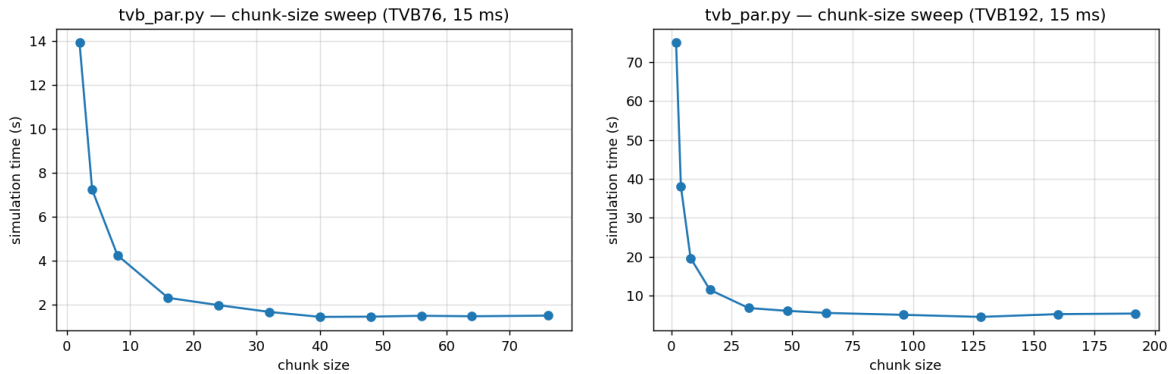


Figure 1: `tvb_par.py` simulation time vs `executor.map` chunk size, single rep, $t_f=15$ ms. The optimum is **chunk ≈ 40 for TVB76** and **chunk ≈ 128 for TVB192** (about $N/2$ in both cases).

In the low region the curve is roughly $1/\text{chunk}$: each task submission has a fixed serialisation and dispatch cost, so packing k centres into one task amortises that cost k times. Past the optimum the curve flattens and starts climbing because the pool runs out of independent work to hand out. With 8 vCPUs and a chunk size of N , you only get $\lceil N/\text{chunk} \rceil$ chunks per timestep, so anything larger than $N/8$ leaves cores idle. The other effect is load imbalance: bigger chunks mean the last (smaller) chunk waits longer at the barrier. The sweet spot is whatever chunk size is big enough to amortise the dispatch cost while still keeping all 8 cores busy.

4. Scaling 15 ms $\rightarrow$ 60 ms (optimal chunk).

Dataset	15 ms (s)	60 ms (s)	Ratio
TVB76 (chunk 40)	1.56	19.34	12.41×
TVB192 (chunk 128)	4.59	58.81	12.81×

Table 2: Wall-clock scaling of the multiprocessing version when tf grows by $4\times$. Min of 3 runs, optimal chunk.

The runtime scales much worse than linear: a $4\times$ jump in tf costs about $12.5\times$ in wall time, not $4\times$. Per-timestep useful work is constant, but X_s grows linearly in T , so the per-task serialised payload grows linearly too. With $T = 300 \rightarrow 1200$, every X_s that gets pickled is $4\times$ bigger and it gets pickled $4\times$ as many times, which puts the IPC cost on a T^2 curve while the useful work stays on T . The measured ratio of ≈ 12.4 matches that prediction fairly closely.

5. *py-spy* profiling at the optimal chunk.

I ran `py-spy record -d 25` against `tvb_par.py` on TVB192 with `chunk=128`. Aggregating samples by function name (the raw flame graph is hard to read because most of the parent’s time is in `_call_with_frames_removed` from initial module imports) gives the runtime/IPC frames in Figure 2.

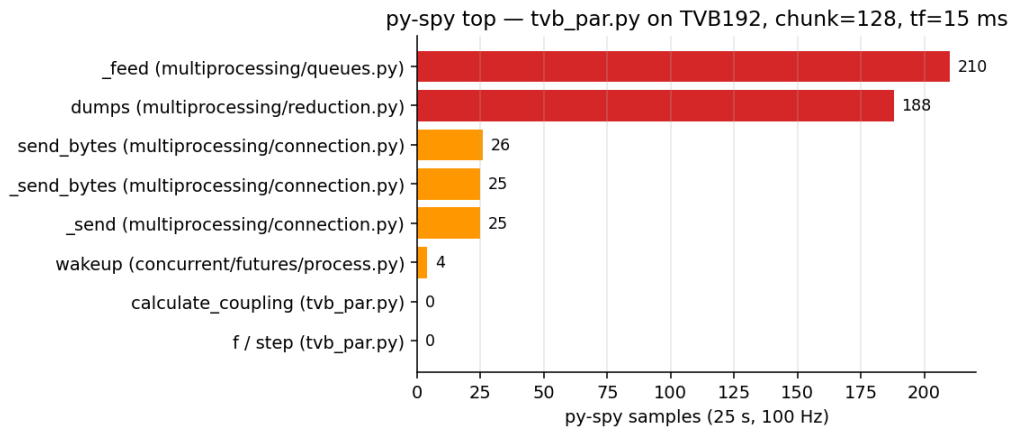


Figure 2: Top runtime frames in the parent process during a `tvb_par.py` run on TVB192 at `chunk=128` ($\text{tf}=15$ ms, 25 s sample at 100 Hz, module-import frames excluded).

The TVB kernels (`f`, `calculate_coupling`, `step`) are not in the parent process at all, they run in the workers. What the parent does spend its time on is `_feed` and `dumps`, which are the multiprocessing queue handler and pickle serialiser, plus the `_send_bytes/_send` chain that pushes the serialised tasks down the pipe to the workers. So the parent process is essentially busy converting nested-list X_s arrays into bytes and writing them into a pipe, all day long. That’s the same diagnosis as Q2.2.2 and motivates shared memory in Q2.3.

Question 2.3 — Shared memory (`tvb_par_sm.py`)

Run on the AWS VM at the optimal chunk found in 2.2.3 (40 for TVB76, 128 for TVB192). For TVB998 I tested chunks $\{32, 64, 128, 256, 512\}$; the best was **64**.

Moving X_s , W and D from nested Python lists to shared-memory NumPy arrays changes what each `executor.map` call has to send: instead of pickling the whole X_s list, each task gets three short names

Dataset	Sequential MLP (s)	tvb_par.py (s)	tvb_par_sm.py (s)	vs tvb_par.py
TVB76	0.77	1.48	1.84	0.81×
TVB192	2.56	6.16	8.54	0.72×
TVB998	N/A	N/A	69.29	—

Table 3: Effect of switching to shared-memory NumPy arrays. Each cell is the minimum of 3 runs at $\text{tf}=15$ ms.

+ dtype/shape tuples and looks up the data by name. Per-message bytes drop by orders of magnitude.

But the wall time on TVB76 and TVB192 actually goes *up* a bit relative to `tvb_par.py` (1.84 s vs 1.48 s on TVB76, 8.54 s vs 6.16 s on TVB192). The reason is that on every task the worker now does a fresh `shared_memory.SharedMemory(name=...)` attach plus three `np.ndarray` view constructions, and the hot inner loops are doing NumPy element indexing (`Xs[i, 0, t-1]`) rather than nested-list indexing, which is slower per access when the per-call work is small. So at small N we trade one overhead for a different overhead.

At TVB998 the per-task overhead is small relative to the $\mathcal{O}(N^2)$ coupling sum and the absolute number lands at 69.3 s. The fix for the per-task SHM attach lives in Q2.4 (`initializer`) and the fix for the slow per-element indexing is the vectorised kernel in Q2.6.

Question 2.4 — Shared memory + initializer (`tvb_par_sm_imp.py`)

Dataset	Sequential MLP (s)	tvb_par_sm.py (s)	tvb_par_sm_imp.py (s)	vs tvb_par_sm.py
TVB76	0.77	1.84	1.25	1.47×
TVB192	2.56	8.54	4.97	1.72×
TVB998	N/A	69.29	44.63	1.55×

Table 4: Effect of moving the shared-memory attach into an `initializer` so it runs once per worker instead of once per task. $\text{tf}=15$ ms, optimal chunk.

Putting the SHM attach into an `initializer` runs it once per worker process instead of once per task. The improvement is visible on every dataset (1.47×, 1.72×, 1.55×). With this change `par_smi` is now slightly faster than the original `tvb_par.py` (1.25 s vs 1.48 s on TVB76, 4.97 s vs 6.16 s on TVB192).

It still loses to plain sequential MLP though (1.25 s vs 0.77 s on TVB76, 4.97 s vs 2.56 s on TVB192), because the worker bodies are the same pure-Python interpreter loops as before, just indexing into NumPy arrays one element at a time. The per-call work is too small for the parallel speedup to pay back the IPC and pool overhead. The fix is to make each worker call do real work, which is what Q2.6 does by vectorising.

Question 2.5 — When does multiprocessing pay off?

Mostly when the per-task work is big enough that the IPC cost is small in comparison. From the numbers above, that means:

- The task body has to be heavy. Pure-Python list-indexing arithmetic per centre per timestep is too

small. Vectorised NumPy is heavy enough.

- There has to be enough independent work between barriers. TVB has one barrier per timestep, but within a timestep all N centres are independent, so larger N helps.
- Shared state has to be either small enough to pickle every time (rarely the case) or live in shared memory. Otherwise the pickle cost wipes out the parallel speedup, see Q2.2.5.
- The work is CPU-bound. For I/O-bound work threads are usually fine because the GIL is released during I/O.
- There are enough cores. The 8-vCPU VM has 4 physical cores, so the realistic ceiling is roughly $4\times$ plus a bit of extra from SMT.

For TVB on this hardware the takeaway is that multiprocessing only actually wins once the inner kernels are vectorised (so the per-task work is big enough) and state lives in shared memory (so the per-task IPC is small). That combination is exactly what L2_Q6 implements.

Question 2.6 — Combined: multiprocessing + sparse + vectorisation + shared memory (L2_Q6.py)

Implementation. I started from `tvb_par_sm_imp.py` (process pool with an `initializer`-attached SHM) and added the two Lab 1 optimisations on top:

- NumPy MLP. The two pure-Python double loops in `f(x, y)` become two matmuls (`np.dot`) plus an in-place `np.maximum` for ReLU.
- Sparse (CSR) coupling. The driver builds `(W_data, D_data, col_idx, row_ptr)` once and puts the arrays into shared memory alongside `Xs`. The worker's coupling sum becomes one `np.sum(W_row * (x_src - 1.0))` over the non-zeros of the row, with a boolean ready mask so that edges whose delay hasn't been reached yet contribute `W * pre(0, ...)` the same way the reference loop does.

Results.

Dataset	<code>tvb_par_sm_imp.py</code> (s)	<code>L2_Q6.py</code> (s)	Speed-up
TVB76	1.25	0.41	3.08$\times$
TVB192	4.97	1.00	4.96$\times$
TVB998	44.63	2.99	14.91$\times$

Table 5: L2_Q6 vs the best provided multiprocess baseline (`tvb_par_sm_imp.py`). `tf=15 ms`, min of 3 runs.

Vectorising the inner kernels gets rid of most of the Python interpreter overhead. CSR turns the dense $\mathcal{O}(N)$ coupling loop into $\mathcal{O}(\text{NNZ})$, where NNZ is the number of non-zero edges (about 5–25% of N^2 for these connectomes). The two effects multiply, which is why the speedup grows with N : TVB76 gets

3.08 $\times$, TVB192 gets 4.96 $\times$, TVB998 gets 14.91 $\times$. That last one matters because TVB998 is where the dense $\mathcal{O}(N^2)$ coupling really hurts, and CSR fixes it.

L2_Q6 also finally beats the sequential MLP at every dataset (0.41 s vs 0.77 s on TVB76, 1.00 s vs 2.56 s on TVB192). Sequential TVB998 was not part of the assignment but L2_Q6 lands at 2.99 s which is comfortably faster than the dense par_smi at 44.63 s.

Validation. I wrote `validate_q6.py` to run the provided `tvb_par_sm_imp.py` and L2_Q6 on TVB76/5 ms and compare the trajectories elementwise. Max absolute difference came out at **3.109e-15**, which is just floating-point reordering noise.

Question 2.7 — Throughput-oriented multiprocessing (L2_Q7.py)

Implementation. Each worker process now runs a complete TVB simulation. The single-process simulation kernel is the same vectorised + sparse code as L2_Q6, just without the inner pool. I submit 200 of these to one `ProcessPoolExecutor`, reusing the same W and D for every simulation as the assignment specifies.

Is shared memory beneficial here? Not really. In 2.3–2.6 the same X_s array was being read and written by 8 cooperating workers T times, so the per-task IPC cost of copying X_s was the bottleneck and shared memory removed it. Here each simulation has its own X_s that no other simulation ever touches, so there is nothing to share. W and D are read-only and are pickled once per task. For TVB192 they total around 0.6 MB, which is small next to the cost of running a 300-timestep simulation. I'd add the SHM machinery for TVB998 (about 16 MB of $W+D$) but for TVB192 it would be added complexity for no measurable speedup.

Results. 200 simulations, TVB192, $\tau_f=15$ ms, single run on the 8-vCPU VM:

Strategy	Wall (s)	Throughput (timesteps/s)
L2_Q7 (parallel-of-sims, vectorised+sparse)	45.12	1330
L2_Q6 back-to-back $\times$ 200	200.42	299

Table 6: Throughput comparison: 200 TVB192 simulations of 15 ms (60,000 total timesteps).

Which performs better and why? L2_Q7 wins. It does 4.44 $\times$ more timesteps per second than running L2_Q6 200 times back-to-back. L2_Q6 has a barrier per timestep and pays fan-out/fan-in costs across 8 workers T times per simulation, and $200 \cdot T$ times across the whole batch. L2_Q7's worker just runs the simulation start to finish without ever talking to the parent again, so the only IPC is one task submission and one return. When the goal is to run a lot of TVB simulations (the parameter-sweep use case mentioned in the assignment intro) the parallel-of-sims layout is the better fit.

Question 2.8 — Multithreading (L2_Q8.py)

1. Multithreaded vs multiprocessed.

The threaded version is faster than the multiprocessed one but still slower than sequential. The kernels

Dataset	Sequential MLP (s)	Multiprocess (s)	Multithread (s)	thr / proc
TVB76	0.77	1.56	1.01	0.65 × (thr 1.54 × faster)
TVB192	2.56	4.59	3.19	0.69 × (thr 1.44 × faster)

Table 7: Threading vs processes vs sequential, $t_f=15$ ms, optimal chunk in each case. The thread pool serialises on the GIL but avoids the per-task IPC bill that multiprocessing pays.

(f, calculate_coupling, step) are list-indexing arithmetic that holds the GIL on almost every line, so threads can't actually run concurrently. All you get is context-switch and GIL-acquisition overhead on top of single-threaded work, which is why the threaded version is slower than sequential. The reason it beats multiprocessing is that the multiprocess pool isn't getting any real parallelism either at this scale (the work per task is too small to amortise IPC), and the threaded version skips the IPC cost since threads share X_s directly.

2. Scaling 15 ms → 60 ms.

Dataset	15 ms (s)	60 ms (s)	Ratio
TVB76 (thr)	1.01	4.59	4.52 ×
TVB192 (thr)	3.19	14.83	4.65 ×

Table 8: Threaded scaling at $t_f=15$ ms vs 60 ms.

Threading scales close to linear ($4.5\times$ – $4.7\times$ for a $4\times$ jump in t_f). Multiprocessing was super-linear at about $12\times$ because the pickled X_s was growing with T . Threads share the same Python list, so there is no growing payload to copy: total work is roughly proportional to the number of timesteps, which is what shows up in the table.

3. Does shared-memory optimisation transfer to threads? No. The whole reason `multiprocessing.shared_memory` exists is so that separate processes can see the same physical memory pages. Threads inside one Python process already share all the memory by default. Wrapping a thread pool around `shared_memory` would just be a no-op layer of indirection. If you want threads to scale on CPU work, the thing to fix is to drop into code that releases the GIL (NumPy, Numba with `nogil=True`, or a C extension).

Question 2.9 — GIL questions

1. What is Python's GIL? The Global Interpreter Lock is one mutex inside CPython that protects the interpreter state. Only one thread per process can run Python bytecode at a time. CPython releases the GIL around blocking I/O calls and around long C extension routines that explicitly drop it (NumPy, parts of SciPy, Numba functions compiled with `nogil=True`). It is held for ordinary Python work like arithmetic, attribute access, and list indexing.

2. Threaded TVB vs sequential. Threaded is slower than sequential (1.01 s vs 0.77 s on TVB76, 3.19 s vs 2.56 s on TVB192). The kernels are pure-Python arithmetic that holds the GIL on practically every line, so threads can't run any of it in parallel. What you do get is the extra cost of switching between threads and acquiring the GIL, which is why the threaded version is slower than just running it on one thread. It still beats multiprocessing because multiprocessing isn't getting real parallelism either at this

problem size, and threads at least don't pay the per-timestep IPC cost of shipping X_s between processes.

3. Can multithreading alone improve sequential TVB? No. The hot path is pure-Python bytecode and that is exactly what the GIL serialises. Threading would only help once the hot path has been moved into code that drops the GIL (NumPy `matmul`, Numba with `nogil=True`, Cython). At that point you usually don't need the threads anyway because one core already runs the C kernel close to memory bandwidth.

4. Where does Python multithreading alone beat sequential? Mostly two cases: I/O-bound work, and CPU-bound work whose hot path is in a library that drops the GIL.

- I/O fan-out: HTTP requests with `requests`, database queries, file reads. The GIL is dropped during the underlying kernel `recv/read`, so k threads can wait in parallel.
- Disk I/O on a fast SSD, for the same reason.
- Code that spends its time inside C/Fortran kernels that drop the GIL: NumPy linear algebra (BLAS), `np.matmul`, `scipy.linalg.solve`, Numba functions with `nogil=True`, image libraries like Pillow and OpenCV.
- `hashlib`, `zlib`, OpenSSL bindings, anything in compression/cryptography.

If the hot path is pure-Python bytecode like the TVB worker functions in `tvb_par.py`, threading on its own can't help because the GIL pins everything to one core. Multiprocessing is the only way to get real parallelism out of that kind of code.

LLM Usage Disclosure

I used Claude (Claude Code) on this assignment for:

- Pair-coding the benchmark harness (`bench.py`, `run_all.sh`) and the chunk-size sweep loop.
- Sanity-checking my GIL answer against the CPython docs and `concurrent.futures` reference.
- Editing this report for clarity (cutting redundant phrasing, reorganising a couple of paragraphs).

I ran every benchmark myself on the assigned AWS VM and read through each of `L2_Q6.py`, `L2_Q7.py`, `L2_Q8.py` before submitting.